TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

DIEGO MAURICIO MIRANDA ZAMBRANO

PARALELO:

"A"

SEMESTRE:

QUINTO

ACTIVIDAD:

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

2. DESARROLLO DE LA ACTIVIDAD PROPUESTA

2.1. Análisis del problema

El Sistema de Control de Rescate y Adopción Animal tiene como finalidad organizar y centralizar la información relacionada con los animales rescatados, desde el reporte inicial hasta su atención veterinaria y posible adopción.

El sistema permitirá registrar animales de diferentes especies, como perros, gatos y fauna silvestre; asignar voluntarios o veterinarios responsables; actualizar los estados de los casos y mantener un historial de las actividades realizadas. De esta manera, se busca mejorar la coordinación del personal, facilitar el seguimiento de cada animal y reducir los errores derivados de una gestión manual de la información.

2.2. Identificación de actores

Se identifican los siguientes actores principales:

Ciudadano/Reportante: registra reportes sobre animales que necesitan rescate y proporciona información sobre su ubicación y condición.

Voluntario/Veterinario: consulta los casos asignados, atiende a los animales, registra las intervenciones realizadas y actualiza los estados correspondientes.

Administrador del Refugio: gestiona los casos, asigna responsables, administra usuarios y supervisa los procesos de rescate y adopción.

2.3. Definición de requerimientos

2.3.1. Requerimientos funcionales

El sistema deberá cumplir las siguientes funciones:

RF01: registrar reportes de animales que necesitan rescate.

RF02: crear un expediente único para cada caso de rescate.

RF03: categorizar los animales según su especie: perro, gato o fauna silvestre.

RF04: almacenar la ubicación, fecha, descripción y demás datos relevantes del caso.

RF05: asignar voluntarios o veterinarios responsables.

RF06: gestionar los estados del proceso, como reportado, rescatado, en tratamiento y en adopción.

RF07: notificar al personal correspondiente sobre nuevos casos.

RF08: registrar las atenciones veterinarias realizadas.

RF09: gestionar las solicitudes y los resultados de adopción.
RF10: consultar el historial de cambios y actividades de cada expediente.

2.3.2. Requerimientos no funcionales

RNF01: escalabilidad, para permitir la incorporación de nuevos usuarios y funcionalidades.
RNF02: mantenibilidad, mediante una arquitectura modular y la aplicación de patrones de diseño.
RNF03: seguridad, mediante el control de acceso según los roles de los usuarios.
RNF04: trazabilidad, para conservar el historial de cambios y las versiones del proyecto.
RNF05: integridad de la información, para evitar registros inconsistentes y adopciones duplicadas.
RNF06: control de versiones, mediante Git, GitHub y el flujo de trabajo Gitflow.

2.4. Diseño del sistema mediante UML

2.4.1. Diagrama de casos de uso

El diagrama de casos de uso representa las interacciones entre los actores y el sistema. El ciudadano puede registrar un reporte; el voluntario o veterinario puede consultar los casos asignados y actualizar la información de atención; y el administrador puede asignar responsables, gestionar expedientes y supervisar los procesos de adopción.
El sistema se representa mediante un límite que contiene los casos de uso, mientras que los actores se ubican en el exterior y se conectan con las funciones que pueden ejecutar.

2.4.2. Diagrama de clases

El diagrama de clases representa la estructura estática del sistema. Se proponen las clases principales `CasoRescate`, `Animal`, `Perro`, `Gato`, `FaunaSilvestre`, `Usuario`, `Voluntario`, `Veterinario`, `Administrador`, `AtencionVeterinaria`, `SolicitudAdopcion` y `Notificador`.
La clase `CasoRescate` administra la información del expediente, mientras que `Animal` contiene los datos generales del animal. Las clases específicas de especie permiten representar características particulares. Las clases de usuario permiten distinguir las responsabilidades de cada participante.
Las relaciones entre estas clases se definirán mediante asociaciones, herencia y multiplicidades UML, procurando mantener una estructura coherente, reutilizable y alineada con los principios SOLID.

2.5. Aplicación de patrones de diseño

2.5.1. Singleton

Este patrón garantiza una única instancia de un componente dentro del contexto de ejecución de la aplicación. Se propone para un gestor central de casos o un componente de configuración compartida, evitando la creación descontrolada de instancias.

Su utilización deberá limitarse a componentes que realmente necesiten una instancia única, sin convertir todas las clases del sistema en objetos globales.

2.5.2. Observer

El patrón Observer permite que un objeto notifique automáticamente a otros objetos interesados cuando ocurre un evento.

En el sistema propuesto, cuando se registra un nuevo caso de rescate, el componente encargado de gestionar los eventos podrá notificar a los voluntarios correspondientes. Esto reduce el acoplamiento entre el registro de casos y el servicio de notificaciones.

2.5.3. Factory Method

Este patrón permite delegar la creación de objetos a métodos especializados. Se utilizará para crear expedientes específicos según la especie del animal: perro, gato o fauna silvestre.

De esta manera, el sistema podrá incorporar nuevos tipos de expedientes con menos modificaciones en el código que coordina la creación de los casos.

2.5.4. MVC (Modelo-Vista-Controlador)

El patrón MVC organiza la aplicación en tres componentes:

Modelo: representa los datos y las reglas del negocio, como animales, expedientes y solicitudes de adopción.

Vista: presenta formularios, listados y estados de los casos al usuario.

Controlador: recibe las acciones de la interfaz y coordina las operaciones correspondientes.

Esta separación facilita el mantenimiento, las pruebas y la evolución independiente de la interfaz y la lógica del sistema.

2.6. Principios SOLID

El diseño tendrá en cuenta los siguientes principios:

Responsabilidad única (SRP): cada clase tendrá una responsabilidad bien definida.

Abierto/cerrado (OCP): se procurará incorporar nuevos tipos de animales sin modificar innecesariamente las clases existentes.

Sustitución de Liskov (LSP): las clases específicas de animales deberán poder utilizarse donde se espere un animal genérico, respetando el comportamiento definido.
Segregación de interfaces (ISP): las interfaces deberán contener operaciones relacionadas con responsabilidades concretas.
Inversión de dependencias (DIP): los componentes principales dependerán de abstracciones para facilitar el reemplazo de implementaciones y las pruebas.

2.7. Gestión del proyecto mediante GitHub y Gitflow

Se utilizará Git para registrar los cambios del diseño y GitHub para alojar el repositorio remoto.
La estructura de ramas contemplará:
- `master`: versión estable preparada para entrega.
- `develop`: integración de los avances del proyecto.
- `feature/analisis`: desarrollo del análisis y los requerimientos.
- `feature/uml`: elaboración de los diagramas UML.
- `feature/patrones`: documentación de los patrones de diseño.

Cada avance se registrará mediante commits con mensajes descriptivos. Las ramas de funcionalidad se integrarán en develop después de revisar sus cambios, y la versión final se incorporará a master cuando el trabajo esté listo.

El repositorio incluirá la documentación, los diagramas y las evidencias necesarias para demostrar el proceso de desarrollo.

2.8. Diseño de la arquitectura general

La arquitectura propuesta estará organizada en capas:

Presentación: interfaz utilizada por los ciudadanos, voluntarios, veterinarios y administradores.

Control de aplicación: controladores que reciben las solicitudes y coordinan las operaciones.

Lógica de negocio: servicios para gestionar rescates, asignaciones, atención veterinaria y adopciones.

Persistencia: componentes responsables de almacenar y recuperar la información.

Notificaciones: componente que comunica los eventos relevantes al personal del refugio.

Los patrones de diseño se integrarán en las capas correspondientes. Factory Method apoyará la creación de expedientes; Observer gestionará las notificaciones por eventos; Singleton se limitará a un componente central que justifique una instancia única; y MVC organizará la interacción entre la presentación, los controladores y el modelo.

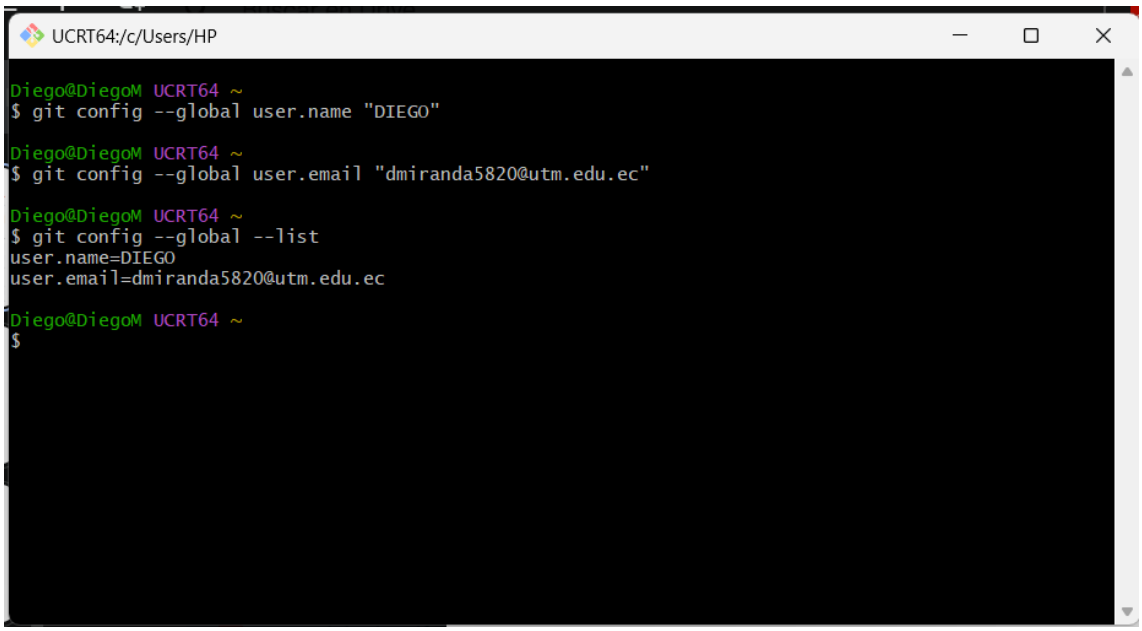
El flujo Gitflow permitirá mantener un historial organizado de los cambios realizados en la documentación y en el diseño de la arquitectura.

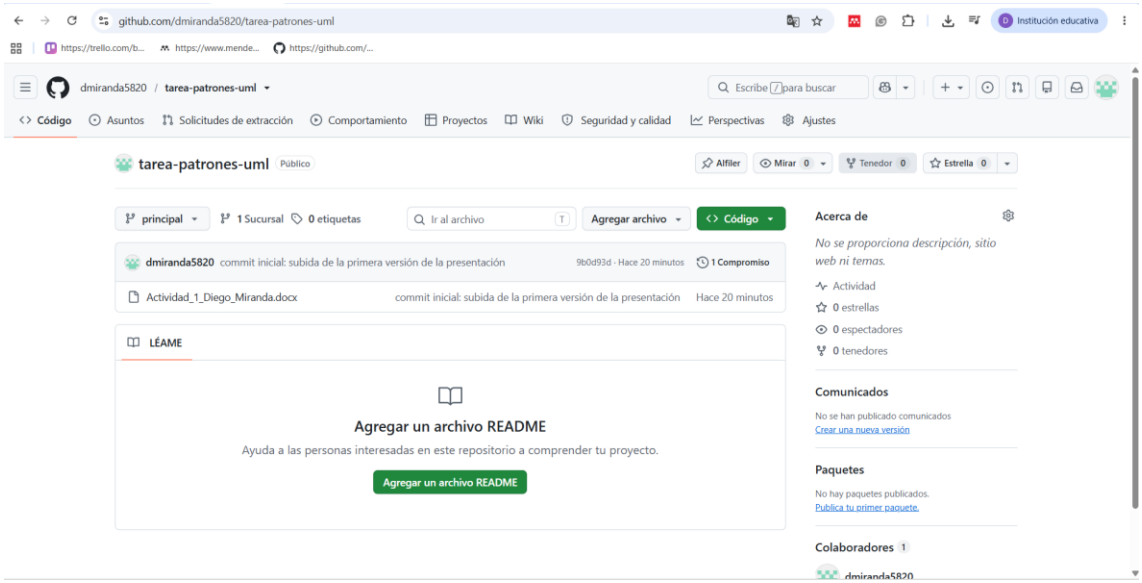
Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).



https://lucid.app/lucidchart/35ad515a-a387-42b7-9e51-76115da00791/edit?viewport_loc=-1070%2C-155%2C3346%2C1660%2C0_0&invitationId=inv_341956dd-d090-4084-9dc2-fd1a1d8ab004





https://github.com/dmiranda5820/tarea-patrones-uml.git



Bibliografía

Astigarraga, J., & Cruz-Alonso, V. (2022). ¡ Se puede entender cómo funcionan Git y GitHub!. ecosistemas, 31(1), 2332-2332.

Prieto, J. G. (2018). ¿ Por qué usar GitHub? Diez pasos para disfrutar de GitHub y no morir en el intento. Ecosistemas, 27(2), 140-141.